

lab03_leila_kawanishi_277185

Leila Kawanishi

Laboratório 3: Dados relacionais e operações do tipo join

Benilton Carvalho, Guilherme Ludwig

Introdução

Conjuntos de dados estruturados costumam ser divididos em múltiplas tabelas. A razão para isso é a minimização de duplicação de informação. Por este motivo, uma tarefa habitualmente realizada em manipulação de bases de dados é a combinação de dados de diferentes origens.

Objetivos

Ao fim deste laboratório, você deve ser capaz de combinar duas tabelas de dados, de forma que, na tabela resultante, sejam mantidos registros que existam:

- em ambas as tabelas;
- apenas na primeira tabela;
- apenas na segunda tabela;
- em alguma das duas tabelas.

Adicionalmente, você deve ser capaz de identificar registros que existam:

- apenas na primeira tabela;
- apenas na segunda tabela;

Carregue os pacotes relevantes para este Laboratório

Atrasos de vôos

Considere novamente o problema de atrasos de vôos, disponível em <https://www.kaggle.com/usdot/flight-delays>. Nesta atividade, além dos dados de `flights.csv`, nós iremos utilizar informações disponíveis nos arquivos `airlines.csv` e `airports.csv`.

1. Importe, utilizando o pacote `readr`, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.

```
library(readr)
library(dplyr)
```

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

`filter`, `lag`

Os seguintes objetos são mascarados por 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
airlines <- read_csv("../airlines.csv")
```

Rows: 14 Columns: 2

```
-- Column specification -----
Delimiter: ","
chr (2): IATA_CODE, AIRLINE
```

i Use ``spec()`` to retrieve the full column specification for this data.

i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

- Para o arquivo de aeroportos, importe apenas as colunas `IATA_CODE`, `CITY`, `STATE`, `LATITUDE`, `LONGITUDE`;

```
airports <- read_csv("../airports.csv",
                     col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE)
                     )
```

Rows: 322 Columns: 5

-- Column specification -----

Delimiter: ","

chr (3): IATA_CODE, CITY, STATE

dbl (2): LATITUDE, LONGITUDE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

- Para o arquivo de vôos:
 - Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY;
 - Leia apenas 1 milhão de linhas por vez;
 - Remova vôos em que o aeroporto de destino comece com a letra 1;
 - Remova registros em que existam pelo menos uma coluna faltante;
 - Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
 - Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
resultados <- list() #armazena as estatisticas de cada bloco

flights_modificado <- function(x, pos) {

  x <- x %>%
    filter(
      !startsWith(DESTINATION_AIRPORT, "1"), #remove aeroporto que começa com a letra 1
      !is.na(DESTINATION_AIRPORT), #filtra NA
      !is.na(ARRIVAL_DELAY) #filtra NA
    )

  estatisticas <- x %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      soma_atrasos = sum(ARRIVAL_DELAY),
      total_voos = n()
    )
}
```

```

    resultados[[length(resultados) + 1]] <- estatisticas
  }

flights <- read_csv_chunked("../flights.csv",
  col_types = cols_only( #importa apenas colunas que quero
    DESTINATION_AIRPORT = col_character(),
    ARRIVAL_DELAY = col_integer()
  ),
  chunk_size = 1000000, #lê apenas 1000000 linhas por bloco
  callback = DataFrameCallback$new(flights_modificado)
)

resultado_final <- bind_rows(resultados) %>%
  group_by(DESTINATION_AIRPORT) %>%
  summarise(
    soma_atrasos = sum(soma_atrasos),
    total_voos = sum(total_voos),
    .groups = "drop") %>%
  mutate(
    atraso_medio = soma_atrasos / total_voos
  )

head(resultado_final)

```

```

# A tibble: 6 x 4
  DESTINATION_AIRPORT soma_atrasos total_voos atraso_medio
  <chr>                <int>        <int>         <dbl>
1 ABE                  12874         2220          5.80
2 ABI                   9077         2224          4.08
3 ABQ                106415        18953          5.61
4 ABR                 -2227          657         -3.39
5 ABY                   7501          864          8.68
6 ACK                   2393          487          4.91

```

2. Selecione a operação apropriada join para incluir, na tabela flights, as colunas CITY e STATE do objeto airports. Para executar esta tarefa:

- Identifique a coluna que é a chave na tabela flights; Resposta: DESTINATION_AIRPORT
- Identifique a coluna que é a chave na tabela airports; Resposta: IATA_CODE

- Quais são os aeroportos que estão listados em flights, mas estão ausentes em airports?
Resposta: Não existem aeroportos listados em flights que estejam ausentes em airports.
- Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
- Armazene a tabela resultante no objeto flights.

```
#cria o objeto unificando as colunas das duas tabelas
flights <- left_join(
  flights,
  airports %>% select(IATA_CODE, CITY, STATE),
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)
```

```
#verifica se existe valores na chave flights ausentes na chave de airports
anti_join(
  flights,
  airports,
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
) %>%
distinct(DESTINATION_AIRPORT)
```

```
# A tibble: 0 x 1
# i 1 variable: DESTINATION_AIRPORT <chr>
```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
airports %>%
  group_by(STATE) %>%
  summarise(
    quantidade_airports = n() #n() conta quantas linhas existem em cada grupo
  ) %>%
  arrange(desc(quantidade_airports)) #ordena de forma descending
```

```
# A tibble: 54 x 2
  STATE quantidade_airports
  <chr>           <int>
1 TX              24
2 CA              22
3 AK              19
4 FL              17
```

```

5 MI 15
6 NY 14
7 CO 10
8 MN 8
9 MT 8
10 NC 8
# i 44 more rows

```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

- Carregue o pacote leaflet;
- Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico;
- Armazene o grafico em b) numa variável chamada this;
- Adicione as duas linhas abaixo após a criação da variável this (apenas se você estiver usando Jupyter):

```

#install.packages("leaflet")
library(leaflet)

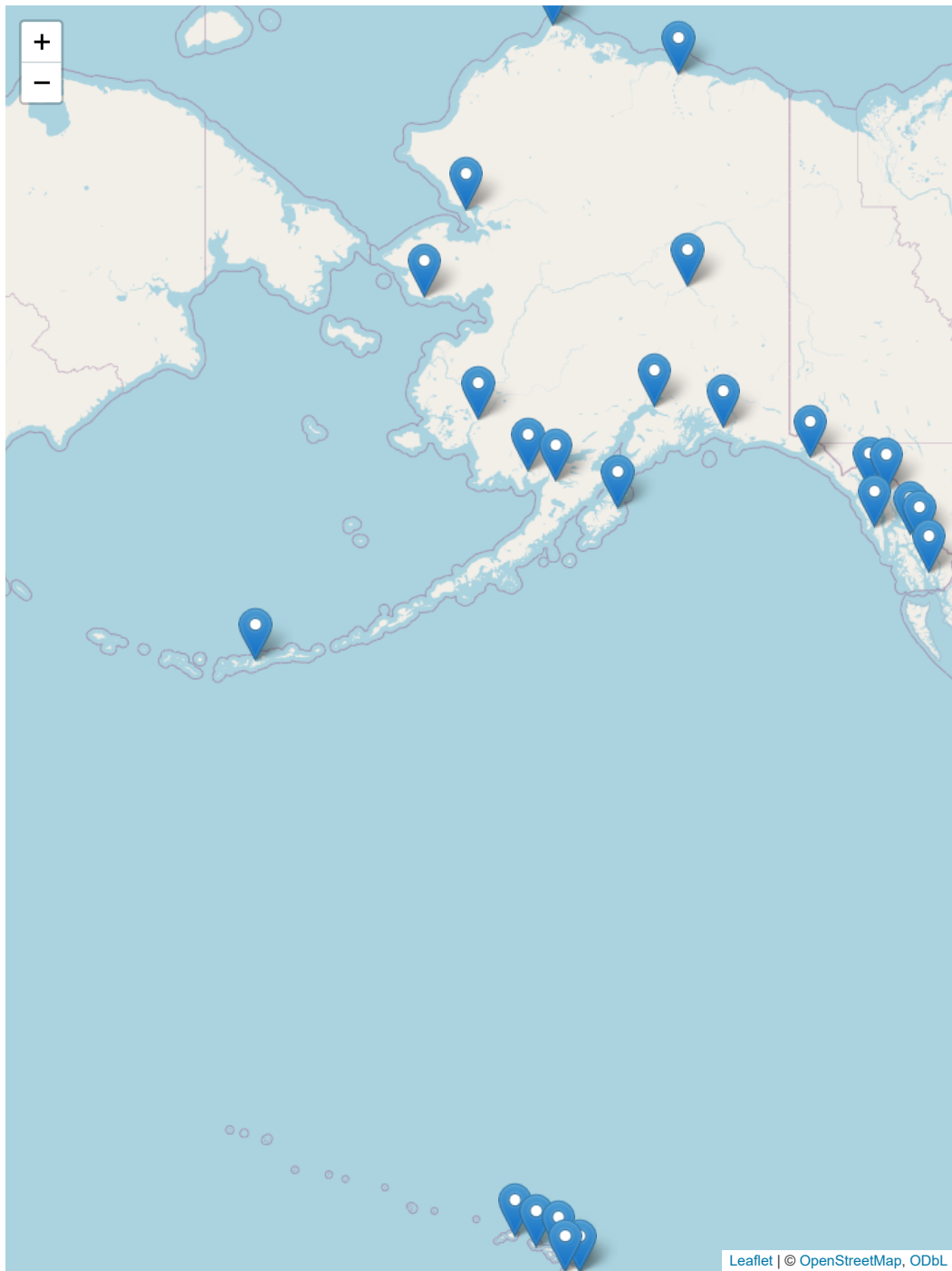
airports_mapa <- left_join(
  airports,
  resultado_final %>%
  select(DESTINATION_AIRPORT, atraso_medio),
  by = c("IATA_CODE" = "DESTINATION_AIRPORT")
)

this <- leaflet(airports_mapa) %>% #vamos trabalhar com essa tabela
  addTiles() %>% #adiciona o mapa_base
  addMarkers( #adiciona marcador em cada aeroporto utilizando sua localização
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~paste0(
      "<b>", CITY, "</b><br>",
      "Estado: ", STATE, "<br>",
      "Atraso médio: ", round(atraso_medio, 2), " minutos"
    )
  )
)

```

Warning in validateCoords(lng, lat, funcName): Data contains 3 rows with either missing or invalid lat/lon values and will be ignored

this




```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 15:09:06 -03"
```